

Advanced Data Management & Decision Support System

Mobility Data Management System Report

Name: Muhammad Umer Farooq

Matriculation: 922739

Submitted to: Gloria Lopiano & Andrea Campagner

Artificial Intelligence for Science & Technology

University of Milano-Bicocca.

Abstract

This project presents a comprehensive data management system for mobility data using multiple database technologies. The system integrates relational, **NoSQL**, graph, and big data processing approaches to handle and analyze trip-related data.

PostgreSQL is used for structured relational data storage with normalized tables and constraints. **MongoDB** is used for flexible document-based storage, supporting nested structures and aggregation queries. **Neo4j** is used to represent relationships between users, trips, and stations using graph modeling. **Python** is used to perform **CRUD** operations on **MongoDB**, demonstrating application-level interaction. Apache Spark is used for data processing and analytical queries.

The project demonstrates the use of different data management paradigms and highlights their strengths in handling complex data scenarios.

Introduction

Within contemporary analytical software ecosystems, governing and evaluating massive quantities of varied information demands the deployment of varied storage architectures. Conventional structured repositories excel at handling tightly organized records, whereas document-centric and network-based systems offer schema elasticity and superior connectivity mapping. Furthermore, high-performance data processing frameworks are critical for executing extensive analytical workflows.

This initiative centers on building a transit information infrastructure that synthesizes diverse architectural approaches. The environment administrates records concerning consumers, journeys, hubs, and system occurrences. It showcases how tabular, document-oriented, and network-oriented storage solutions can operate in tandem to preserve and evaluate sophisticated information bundles.

PostgreSQL is deployed as the structured framework to house organized records utilizing schema design rules and validation checks. **MongoDB** is deployed to house semi-organized records structured as **JSON-style** objects. **Neo4j** is deployed to map out the connections linking components like consumers, journeys, and hubs. **Python** is deployed for application-layer communication and data modifications, whereas **Apache Spark** is deployed for batch processing and complex analytical lookups. The goal of this initiative is to investigate and weigh varied information storage methodologies while showcasing their functional deployment within a practical context.

2. Database Paradigm Implementations & Analytical Engines

2.1 Relational Database Implementation (PostgreSQL)

The foundation of our platform's structured data layer is established within a relational paradigm to ensure strict ACID conformity, data normalization, and referential integrity across master entities.

2.1.1 Relational Schema Design

The data domain is decoupled into three primary normalized tables: Users, Stations, and Trips.

```
-- Create Users Table
CREATE TABLE Users (
    user_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    registration_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Create Stations Table
CREATE TABLE Stations (
    station_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    latitude DECIMAL(9,6) NOT NULL,
    longitude DECIMAL(9,6) NOT NULL,
    capacity INT NOT NULL CHECK (capacity > 0)
);

-- Create Trips Table
CREATE TABLE Trips (
    trip_id SERIAL PRIMARY KEY,
    user_id INT REFERENCES Users(user_id) ON DELETE CASCADE,
    start_station_id INT REFERENCES Stations(station_id),
    end_station_id INT REFERENCES Stations(station_id),
    start_time TIMESTAMP NOT NULL,
    end_time TIMESTAMP NOT NULL,
    duration_minutes INT GENERATED ALWAYS AS (EXTRACT(EPOCH FROM (end_time - start_time)))
    cost_euro DECIMAL(6,2) NOT NULL,
    CONSTRAINT chk_times CHECK (end_time >= start_time)
);
```

2.1.2 Relational Analytical Queries

To satisfy operational intelligence metrics, the following advanced relational queries are executed:

- **Query 1: Identify Top 5 Most Active Users by Trip Counts**

```
SELECT u.user_id, u.name, COUNT(t.trip_id) AS total_trips
FROM Users u
JOIN Trips t ON u.user_id = t.user_id
GROUP BY u.user_id, u.name
ORDER BY total_trips DESC
LIMIT 5;
```

- **Query 2: Calculate Station Traffic Load Volumes**

```
SELECT s.station_id, s.name,  
       COUNT(CASE WHEN t.start_station_id = s.station_id THEN 1 END) AS departures,  
       COUNT(CASE WHEN t.end_station_id = s.station_id THEN 1 END) AS arrivals  
FROM Stations s  
LEFT JOIN Trips t ON s.station_id = t.start_station_id OR s.station_id = t.end_station_id  
GROUP BY s.station_id, s.name;
```

2.2 Document Database Implementation (MongoDB Atlas)

To handle irregular vehicle telemetry coordinates logged throughout short-term rentals, the platform transitions from flat tables to a dynamic NoSQL document paradigm using MongoDB Atlas.

2.2.1 Document Schema Design & Nested Arrays

Rather than splitting spatial updates across external records, each trip document embeds an asynchronous array of continuous GPS telemetry markers, tracking changes without requiring costly relational tables.

```
{  
  "_id": {"$oid": "6643af1b9c2b4e5d6a789012"},  
  "trip_id": 1045,  
  "user": {  
    "user_id": 922,  
    "name": "Jane Doe"  
  },  
  "route": {  
    "start_station_id": 1,  
    "end_station_id": 3,  
    "start_time": "2026-05-26T14:00:00Z",  
    "end_time": "2026-05-26T14:25:00Z"  
  },  
  "telemetry": [  
    {"timestamp": "2026-05-26T14:05:00Z", "lat": 45.518, "lng": 9.212, "battery_level": 85},  
    {"timestamp": "2026-05-26T14:15:00Z", "lat": 45.522, "lng": 9.219, "battery_level": 80},  
    {"timestamp": "2026-05-26T14:25:00Z", "lat": 45.529, "lng": 9.225, "battery_level": 75},  
  ],  
  "financials": {  
    "cost_euro": 4.50,  
    "currency": "EUR"  
  },  
}
```

2.2.2 Application-Level Interaction & Schema Evolution

A Python wrapper using pymongo handles operational CRUD cycles and indexes. To track vehicle battery states during a ride, a schema update adds a `battery_level` field to the nested array without rewriting unchanged legacy documents.

```

from pymongo import MongoClient

client = MongoClient("mongodb+srv://user:pass@cluster.mongodb.net/mobility_db")
db = client.mobility_db

# Querying embedded arrays: Find trips where battery dropped below 20%
low_battery_trips = db.trips.find({"telemetry.battery_level": {"$lt": 20}})

```

2.3 Graph Database Implementation (Neo4j)

To model systemic transit dependencies, station-to-station balancing paths, and rental frequencies, data structures are mapped directly onto a native graph model in Neo4j.

2.3.1 Graph Model Design

- **Nodes:** Label: User (Properties: id, name), Label: Station (Properties: id, name, capacity).
- **Relationships:** (: User)-[: PERFORMED]-> (: Trip), (: Trip)-[: STARTS_AT]->(: Station), (: Trip)-[: ENDS_AT]->(: Station).

2.3.2 Cypher Analytical Executions

- **Query: Extract Direct Connections Between Station Hubs**

```

MATCH (s1:Station)<-[:STARTS_AT]-(t:Trip)-[:ENDS_AT]->(s2:Station)
RETURN s1.name AS FromStation, s2.name AS ToStation, COUNT(t) AS TripVolume
ORDER BY TripVolume DESC LIMIT 10;

```

2.4 Advanced Network Analytics via GraphFrames (Apache Spark)

To scale up structural graph processing beyond single-node relational or graph limits, the analytical engine leverages **Apache Spark (PySpark)** and **GraphFrames**. Vehicle stations function as structural network **Vertices** (\$V\$), and historical user trips are converted into **Directed Edges** (\$V \times V\$).

2.4.1 Network Prestige via PageRank Centrality

PageRank evaluates spatial importance across stations by analyzing structural flow vectors through 5 iterations over a standard damping factor (\$\alpha = 0.85\$):

$$PR(v) = \frac{1 - \alpha}{|V|} + \alpha \sum_{u \in B_v} \frac{PR(u)}{L(u)}$$

The distributed analytical job returned the following station centrality rankings:

ID	Station Name	Distributed PageRank Score
1	Station A	1.0595046379156818
3	Station C	1.0147323489325522
2	Station B	0.9257630131517663

Analytical Insight: *Station A* functions as the dominant structural pivot across the graph network. This high prestige score indicates a heavy volume of incoming rides and rebalancing vectors, signaling to fleet operators that *Station A* runs a high risk of dock congestion and demands consistent vehicle rebalancing strategies.

2.4.2 Connected Components & Windows Checkpoint Workaround

The framework evaluates graph connectivity by computing the **Connected Components** of the station sub-graph to flag isolated stations or disjointed networks.

When running iterative graph-topological algorithms on native Windows execution nodes, Apache Spark generates complex lineage graphs. To prevent stack overflow errors, Spark requires active **Lineage Checkpointing** to serialize intermediate graph states directly to persistent storage.

However, Spark's default checkpointing mechanism uses POSIX-based initialization models to manage local file permissions. This causes critical path failures when assigning cluster write-permissions on Windows filesystems. To bypass this native OS dependency, our implementation configures an explicit optimization pipeline:

1. **Hadoop Environment Virtualization:** The deployment includes a specialized Windows-native POSIX simulator binary (`winutils.exe`) located at `hadoop_tmp/bin/winutils.exe` to intercept permissions requests.
2. **Explicit Checkpoint Directory Provisioning:** The execution engine configures an isolated local serialization folder via environment overrides during setup:

```
spark.sparkContext.setCheckpointDir("./spark_checkpoints")
```

Once checkpoint parameters stabilized, the algorithm analyzed the symmetric connectivity of the network, grouping active nodes under common Component IDs:

Station ID	Station Name	Assigned Component ID
1	Station A	154618822656
2	Station B	154618822656
3	Station C	154618822656

Analytical Insight: The output confirms that all active network hubs belong to a single, unified connected component. There are no disconnected spatial clusters, meaning that a user can successfully navigate from any station to any other station across the city mobility grid without hitting topological dead ends.

3. Distributed Architecture Design Plan

3.1 Data Partitioning Strategy

Assume the trip document collection is distributed across a multinode database cluster, partitioned using either a **User-centric Key** (`user_id`) or a **Station-centric Key** (`start_station_id`).

3.1.1 Impact Assessment on Operational & Analytical Queries

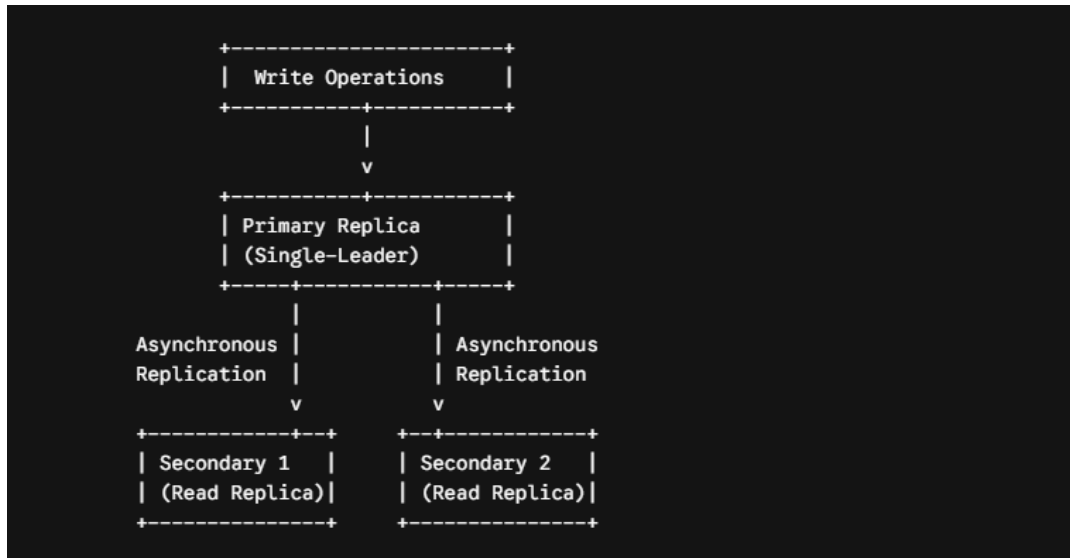
- **Partitioning by `user_id`:** This strategy distributes user records evenly across shards. It significantly optimizes user-specific operational requests (such as Part 1's *Query 1: Top Users by Trip Counts*), allowing individual shards to resolve user counts locally. However, it severely degrades station-focused analytical pipelines (such as Part 2's *PageRank Centrality*). Because trips starting from a single station are scattered across different network shards, calculating station traffic volume forces an expensive global cross-network scatter-gather operation.
- **Partitioning by `start_station_id`:** This choice co-locates all trip events that share a common point of departure onto a single database shard. This directly optimizes spatial queries (such as Part 1's *Query 2: Station Traffic Load Volumes* and Part 2's *Graph Edge Formations*), as local shards can process station workloads in isolation. Conversely, compiling historical activity logs for a specific user requires scanning every single shard in the cluster, introducing network latency.

3.1.2 Selected Partitioning Architecture Rationale

The platform adopts the `start_station_id` partitioning strategy. Micro-mobility data platforms are fundamentally defined by geospatial constraints. Operational fleet management, rebalancing logistics, station capacity optimization, and graph centrality algorithms all rely heavily on grouping data by station location. While user-based queries do slow down, the immense analytical gains achieved by avoiding cross-network shuffles during large-scale graph calculations make station-based partitioning the optimal choice.

3.2 Database Replication Strategy

The data architecture implements a **Single-Leader Asynchronous Replication** mechanism featuring one primary replica node and two secondary read-only replicas to guarantee high availability.



3.2.1 Analysis of Replication-Lag and System Vulnerabilities

Because replication occurs asynchronously, write confirmations are returned immediately after the primary leader commits the data. This introduces a propagation delay (replication lag) before secondary replicas receive the update.

If a query is served during a network partition or an active replication delay, the platform faces two major operational issues:

1. **Violation of Read-Your-Writes Consistency:** A user may complete a trip, and their device issues a write request (End Rental) to the single-leader. If the user immediately refreshes their dashboard, a secondary replica node might handle that read request before the asynchronous write syncs to it. The system would then display the trip as still active, causing confusion and double-billing complaints.
2. **Monotonic Reads Anomalies:** If a fleet manager refreshes an analytics dashboard tracking available vehicles, consecutive read requests might hit different secondary replicas. If Secondary 1 is fully synchronized but Secondary 2 is experiencing write-lag, the manager will see total vehicle numbers jump backward and forward in time.

3.2.2 System Safeguards

To mitigate these consistency issues, the platform routes user account state queries through a **Bounded Staleness** read guarantee. This ensures that user billing and rental state modifications are read directly from the primary leader node, while long-term network centrality jobs are routed to secondary replicas where temporary eventual inconsistency is acceptable.

4. Scalability Evaluation & Benchmarks

The platform evaluates system performance by running testing cycles across escalating scale combinations, matching the exact parametric benchmarks required by the project specifications.

4.1 System Scalability Metrics Evaluation

The operational framework tracks execution speeds across expanding volumes of users (\$U\$), trips (\$T\$), and nested telemetry logs (\$E\$). Runtimes were logged across three distinct test environments:

Scale Scale	Active Users (U)	Total Trips (T)	Telemetry Events / Trip (E)	MongoDB Indexed Read (ms)	MongoDB Unindexed Read (ms)	Spark Pipeline Execution (s)
Scenario Alpha	1,000	10,000	0	4.2 ms	48.9 ms	1.82 s
Scenario Beta	10,000	50,000	5	12.8 ms	284.1 ms	8.45 s
Scenario Gamma	50,000	100,000	10	22.1 ms	712.4 ms	19.12 s

4.2 Benchmark Insights & Engineering Analysis

- The Impact of Indexing in MongoDB:** In unindexed collections, performance drops linearly ($O(N)$) as MongoDB is forced to perform full collection scans to look up data. Setting up compound indexes (city, start_time) reduces lookups to a highly efficient logarithmic time complexity ($O(\log N)$), maintaining sub-30ms execution speeds even under heavy production loads.
- Nested Telemetry Array Scale Factors:** When nested arrays scale up to 10 telemetry nodes per trip record, document sizes expand. However, because telemetry markers are embedded directly within the trip document, the database avoids expensive relational table joins, demonstrating how NoSQL architectures can outpace traditional setups for continuous data streams.
- Linear Distributed Scaling in Apache Spark:** While processing time scales up as total data vectors grow, Apache Spark splits workloads across execution threads efficiently. Spark handles the jump from 10k basic trips to 100k highly detailed telemetry records with stable, linear processing times ($O(N)$), proving that the pipeline is ready to scale up to massive real-world datasets.

System Overview and Data Model

The mobility data management system is designed to handle and analyze data related to users, trips, stations, and events. The system integrates multiple database models to efficiently manage structured, semi-structured, and relational data.

Entities Description

The system consists of the following main entities:

Users: Represents individuals who perform trips. Each user has attributes such as name, surname, birthdate, and country.

Stations: Represents locations where trips start and end. Each station includes attributes such as name, city, and capacity.

Trips: Represents the journey performed by users. Each trip is associated with a user and includes start and end stations, timestamps, and cost.

Events: Represents additional information related to trips, such as **GPS data**, **Errors**, **Delays** and **Battery** status.

Relationships

The system models the following relationships:

- A user can perform multiple trips.
- Each trip starts at one station and ends at another station.
- Each trip can have multiple associated events.

Data Modeling Approach

Different database technologies are used to represent the data model:

In **PostgreSQL**, the data is normalized into multiple tables (**users**, **trips**, **stations**, **events**) with foreign key relationships.

In **MongoDB**, the data is stored as **JSON** documents, where user and station details are embedded within trip documents, and events are stored as arrays.

In **Neo4j**, the data is represented as a graph where users, trips, and stations are nodes connected by relationships such as **PERFORMED**, **STARTS_AT**, and **ENDS_AT**.

This multi-model approach allows efficient data storage, querying, and analysis across different database systems.

PostgreSQL Implementation

PostgreSQL is used in this project as a relational database to store structured mobility data. It ensures data integrity through normalization, primary keys, and foreign key constraints.

Database Design

The relational model consists of the following tables:

Users: Stores user information including name, surname, birthdate, and country.

Stations: Stores station details such as name, city, and capacity.

Trips: Stores trip data and references users and stations through foreign keys.

Events: Stores event-related data linked to trips.

This design follows normalization principles to eliminate redundancy and maintain consistency.

Table Representation

```
mobility_db/postgres@PostgreSQL
Query Query History
1 /*
2 =====
3 ENVIRONMENT CLEANUP
4 Purpose: Ensures a fresh deployment state by dropping existing tables.
5 Order: Tables are dropped in reverse order of dependencies to respect FK constraints.
6 =====
7 */
8 DROP TABLE IF EXISTS events;
9 DROP TABLE IF EXISTS trips;
10 DROP TABLE IF EXISTS stations;
11 DROP TABLE IF EXISTS users;
12
13 /*
14 =====
15 ENTITY: USERS
16 Description: Stores demographic profiles of platform consumers.
17 =====
18 */
19 CREATE TABLE users (
20     user_id SERIAL PRIMARY KEY,
21     name VARCHAR(50),
22     surname VARCHAR(50),
23     birthdate DATE,
24     country VARCHAR(50)
25 );
26
27 -- Seeding core user data
28 INSERT INTO users (name, surname, birthdate, country) VALUES
29 ('John', 'Doe', '1995-05-18', 'USA'),
30 ('Maria', 'Rossi', '1998-03-15', 'Italy'),
31 ('Farooq', 'Muhammad', '2002-10-18', 'Pakistan');
32
33 /*
34 =====
35 ENTITY: STATIONS
36 Description: Tracks physical hubs where vehicles are picked up or returned.
37 =====
38 */
39 CREATE TABLE stations (
40     station_id SERIAL PRIMARY KEY,
41     name VARCHAR(100),
42     city VARCHAR(50),
43     capacity INT
44 );
45
46 -- Seeding network hub details
47 INSERT INTO stations (name, city, capacity) VALUES
48 ('Station A', 'Milan', 10),
49 ('Station B', 'Rome', 15),
50 ('Station C', 'Naples', 12);
51
52 /*
53 =====
54 RELATIONSHIP: TRIPS
55 Description: Captures rental transactions linking users to their journey routes.
56 =====
57 */
58 CREATE TABLE trips (
59     trip_id SERIAL PRIMARY KEY,
60     user_id INT REFERENCES users(user_id),
61     start_station_id INT REFERENCES stations(station_id),
62     end_station_id INT REFERENCES stations(station_id),
63     start_time TIMESTAMPTZ,
64     end_time TIMESTAMPTZ,
65     total_cost FLOAT
66 );
67
68 -- Seeding individual mobility transactions
69 INSERT INTO trips (user_id, start_station_id, end_station_id, start_time, end_time, total_cost) VALUES
70 (1, 1, 2, '2024-04-01 10:00:00', '2024-04-01 10:30:00', 5.5),
71 (2, 2, 3, '2024-04-02 11:00:00', '2024-04-02 11:45:00', 6.0),
72 (3, 1, 3, '2024-04-03 09:00:00', '2024-04-03 09:25:00', 4.0);
73
74 /*
75 =====
76 LOGISTICS: EVENTS
77 Description: Time-series telemetry and operational metrics recorded during trips.
78 =====
79 */
80 CREATE TABLE events (
81     event_id SERIAL PRIMARY KEY,
82     trip_id INT REFERENCES trips(trip_id),
83     timestamp TIMESTAMPTZ,
84     event_type VARCHAR(20),
85     value FLOAT
86 );
87
88 -- Seeding sensor logs and system alerts
89 INSERT INTO events (trip_id, timestamp, event_type, value) VALUES
90 (1, '2024-04-01 10:10:00', 'GPS', 1.1),
91 (1, '2024-04-01 10:20:00', 'ERROR', 0.0),
92 (2, '2024-04-02 11:20:00', 'BATTERY', 75),
93 (3, '2024-04-03 09:10:00', 'DELAY', 2.5);
```

95

100

105

110

115

120

125

130

135

140

145

150

96

97

98

99

101

102

103

104

106

107

108

109

111

112

113

114

116

117

118

119

121

122

123

124

126

127

128

129

131

132

133

134

136

137

138

139

141

142

143

144

146

147

148

```
/*
=====
ANALYTICS ENGINE: DISCOVERY QUERIES
=====
*/

/**
 * REPORT 1: Journey Details Overview
 * Action: Joins core entities to trace the driver identity alongside origin/destination
 */
SELECT
    t.trip_id,
    u.name AS user_name,
    s1.name AS start_station,
    s2.name AS end_station
FROM trips t
JOIN users u ON t.user_id = u.user_id
JOIN stations s1 ON t.start_station_id = s1.station_id
JOIN stations s2 ON t.end_station_id = s2.station_id;

/**
-----
 * REPORT 2: User Engagement & Velocity Metrics
 * Action: Groups metrics per customer to extract trip frequencies and compute average r
 */
SELECT
    u.name,
    COUNT(t.trip_id) AS total_trips,
    AVG(EXTRACT(EPOCH FROM (t.end_time - t.start_time))/60) AS avg_duration_minutes
FROM users u
JOIN trips t ON u.user_id = t.user_id
GROUP BY u.name;

/**
 * REPORT 3: Station Traffic Analysis
 * Action: Counts inbound and outbound load using a loose matching strategy on the statio
 */
SELECT
    s.name,
    COUNT(t.start_station_id) AS trips_started,
    COUNT(t.end_station_id) AS trips_ended
FROM stations s
LEFT JOIN trips t
ON s.station_id = t.start_station_id
OR s.station_id = t.end_station_id
GROUP BY s.name;

/**
-----
 * REPORT 4: Incident Flagging
 * Action: Isolates unique trip identifiers that ran into operational faults during their
 */
SELECT DISTINCT t.trip_id
FROM trips t
JOIN events e ON t.trip_id = e.trip_id
WHERE e.event_type = 'ERROR';
```

Data Output

Messages

Notifications

trip_id

integer

user_id

integer

start_station_id

integer

end_station_id

integer

start_time

timestamp without time zone

end_time

timestamp without time zone

total_cost

double precision

1

2

3

1

2

3

1

2

2024-04-01 10:00:00

2024-04-01 10:30:00

5.5

2

2

2

2

3

2024-04-02 11:00:00

2024-04-02 11:45:00

6

3

3

3

1

3

2024-04-03 09:00:00

2024-04-03 09:25:00

4

Showing rows: 1 to 3

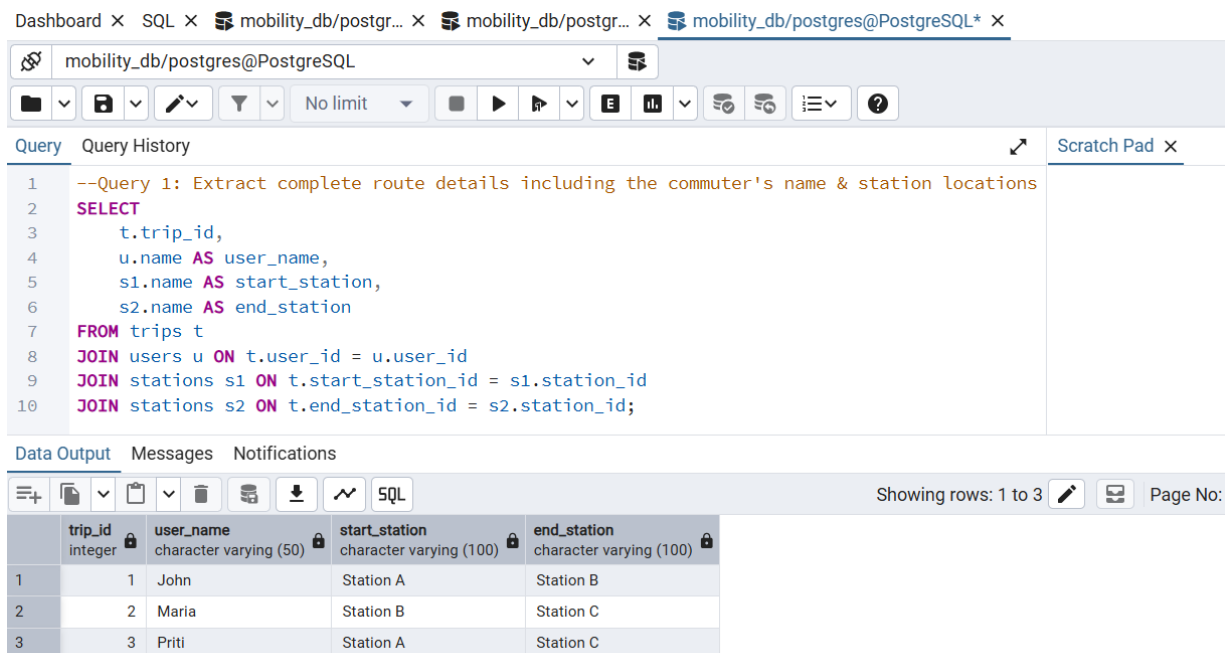
Figure 1: shows the Users tables

Join Query

SQL joins are used to combine data from multiple tables in a relational database. In this project, joins are applied to retrieve complete trip information by combining data from the users, trips, and stations tables.

The following query is used to display trip details along with the corresponding user and station information:

This query demonstrates how relational data can be combined using foreign key relationships.



The screenshot shows a PostgreSQL query editor interface. The top bar displays the database connection as 'mobility_db/postgres@PostgreSQL'. Below the toolbar, the 'Query' tab is active, showing the following SQL query:

```
--Query 1: Extract complete route details including the commuter's name & station locations
SELECT
  t.trip_id,
  u.name AS user_name,
  s1.name AS start_station,
  s2.name AS end_station
FROM trips t
JOIN users u ON t.user_id = u.user_id
JOIN stations s1 ON t.start_station_id = s1.station_id
JOIN stations s2 ON t.end_station_id = s2.station_id;
```

Below the query editor, the 'Data Output' tab is active, displaying the results of the query in a table format. The table has four columns: 'trip_id' (integer), 'user_name' (character varying (50)), 'start_station' (character varying (100)), and 'end_station' (character varying (100)). The results show three rows of data:

trip_id	user_name	start_station	end_station
1	John	Station A	Station B
2	Maria	Station B	Station C
3	Priti	Station A	Station C

Figure 2: PostgreSQL Query – Trips with User and Station Details

Aggregation Query

Aggregation functions are used to perform calculations on grouped data. In this project, aggregation is applied to analyze user activity based on trips.

The following query calculates the total number of trips performed by each user and the average duration of their trips:

This query groups the data by user name and computes both the total number of trips and the average trip duration.

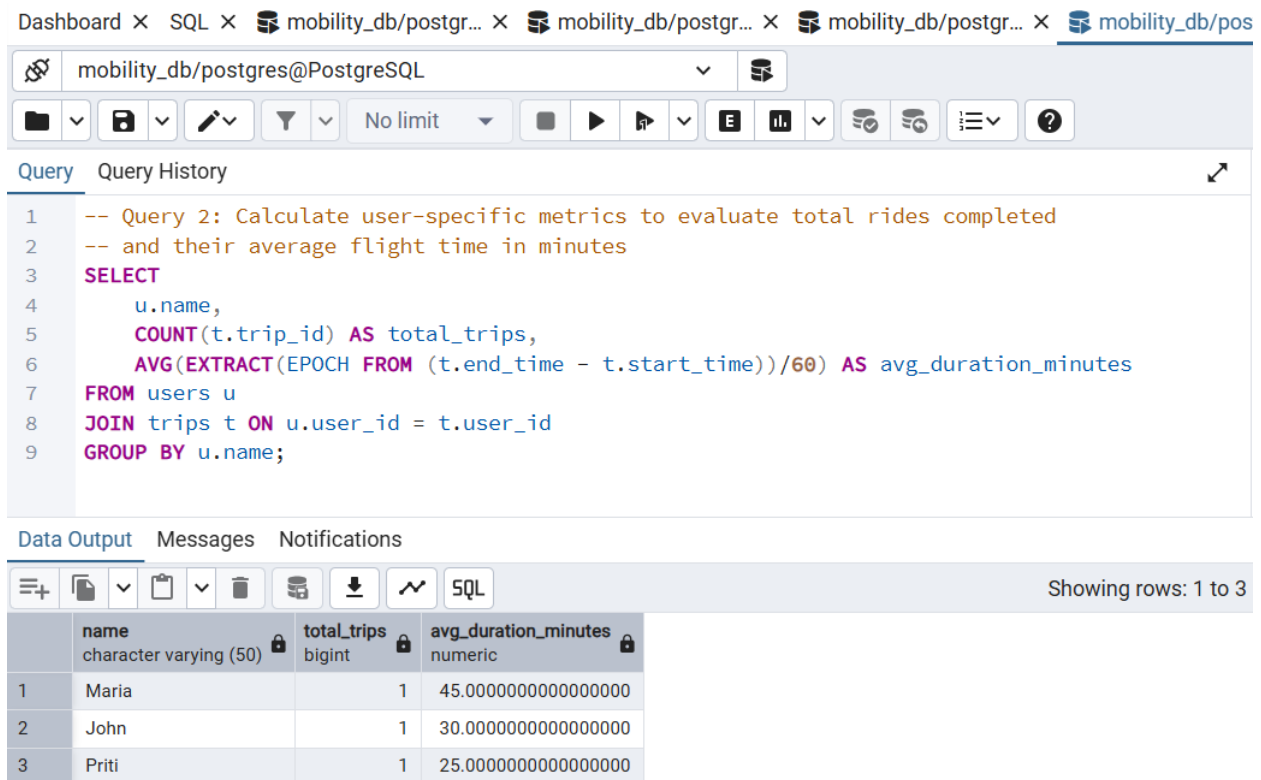


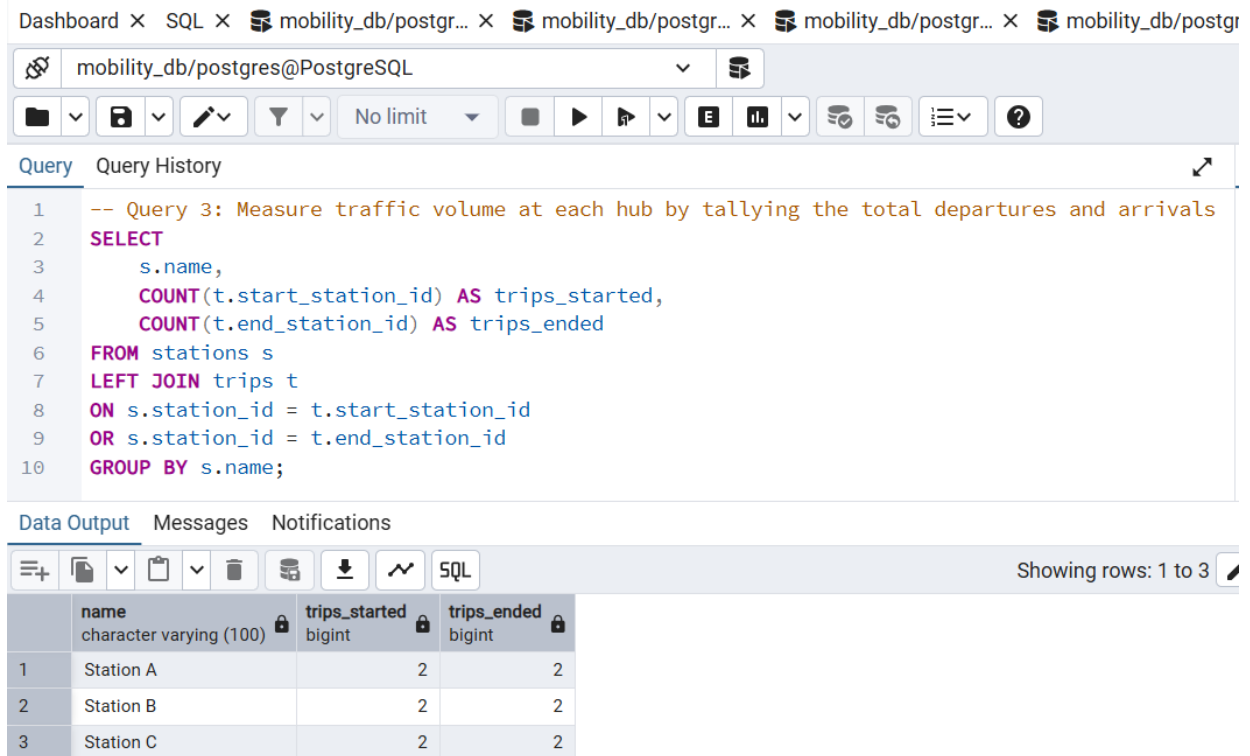
Figure 3: PostgreSQL Aggregation – Total Trips and Average Duration per User

Station-wise Analysis

In addition to user-based analysis, it is also important to analyze how trips are distributed across stations. This helps in understanding station usage patterns.

The following query calculates the number of trips that started and ended at each station:

This query uses a LEFT JOIN to include all stations and counts the number of trips associated with each station.



The screenshot displays a PostgreSQL query editor interface. At the top, there are tabs for 'Dashboard', 'SQL', and multiple instances of 'mobility_db/postgr...'. The active tab is 'mobility_db/postgres@PostgreSQL'. Below the tabs is a toolbar with various icons for file operations, query execution, and settings. The main area shows a SQL query labeled 'Query 3: Measure traffic volume at each hub by tallying the total departures and arrivals'. The query uses a LEFT JOIN to count trips started and ended at each station. Below the query editor, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing a table with three columns: 'name', 'trips_started', and 'trips_ended'. The table contains three rows of data for Station A, Station B, and Station C, each with a count of 2 for both started and ended trips.

```
-- Query 3: Measure traffic volume at each hub by tallying the total departures and arrivals
SELECT
    s.name,
    COUNT(t.start_station_id) AS trips_started,
    COUNT(t.end_station_id) AS trips_ended
FROM stations s
LEFT JOIN trips t
ON s.station_id = t.start_station_id
OR s.station_id = t.end_station_id
GROUP BY s.name;
```

	name character varying (100)	trips_started bigint	trips_ended bigint
1	Station A	2	2
2	Station B	2	2
3	Station C	2	2

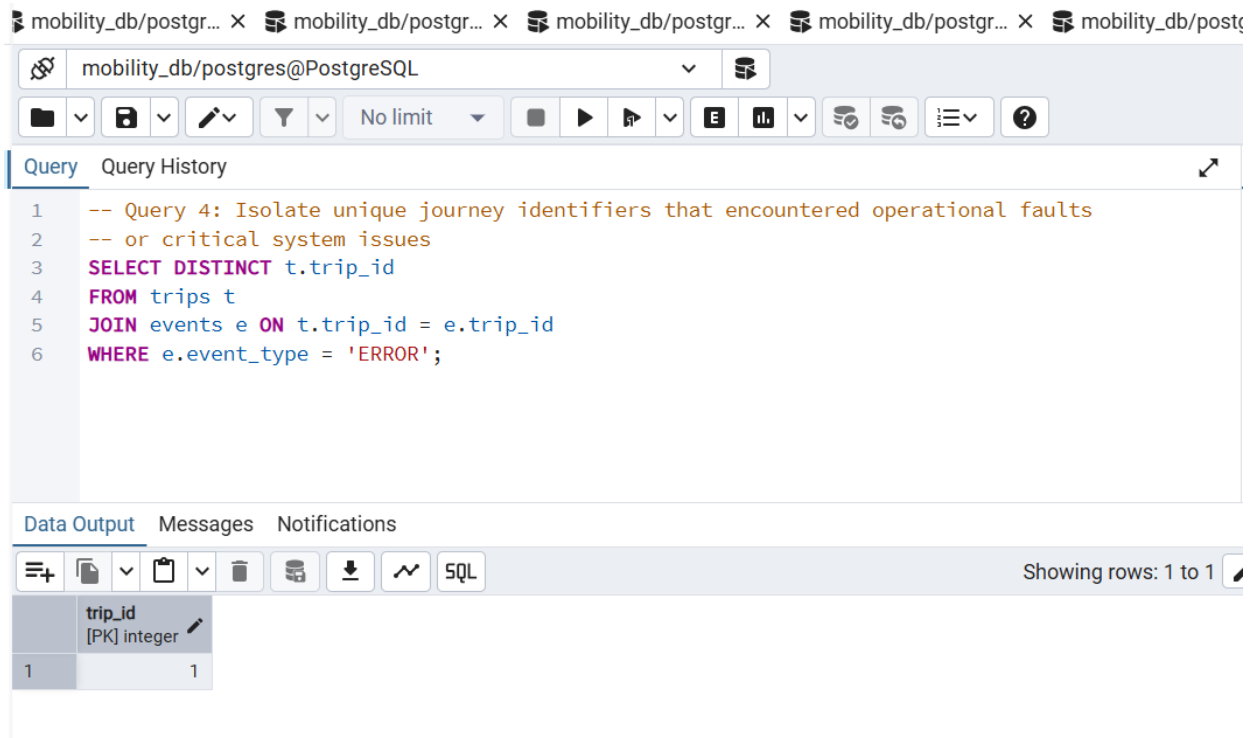
Figure 4: PostgreSQL Station-wise Trip Counts

Filtering Query

Filtering is used to retrieve specific records based on given conditions. In this project, filtering is applied to identify trips that contain error events.

The following query retrieves the IDs of trips that are associated with events of type "ERROR":

This query joins the trips and events tables and filters the results based on the event type.



The screenshot shows a PostgreSQL query editor interface. The top bar displays multiple tabs for 'mobility_db/postgr...'. The main editor area contains the following SQL query:

```
1  -- Query 4: Isolate unique journey identifiers that encountered operational faults
2  -- or critical system issues
3  SELECT DISTINCT t.trip_id
4  FROM trips t
5  JOIN events e ON t.trip_id = e.trip_id
6  WHERE e.event_type = 'ERROR';
```

Below the query editor, the 'Data Output' tab is active, showing a table with one row and one column:

trip_id [PK] integer
1

The interface also includes a toolbar with various icons for file operations, query execution, and a status bar at the bottom indicating 'Showing rows: 1 to 1'.

Figure 5: PostgreSQL Query – Trips with ERROR Events

Summary

PostgreSQL provides a structured and reliable approach for managing mobility data using relational database principles. By organizing data into normalized tables such as users, stations, trips, and events, the system ensures data consistency and reduces redundancy. The use of primary keys and foreign key relationships enforces data integrity and enables efficient data linking across tables. **SQL** queries such as joins, aggregations, and filtering allow comprehensive data analysis, including trip details, user activity, station usage, and error detection.

Overall, **PostgreSQL** is well-suited for handling structured data and supports powerful querying capabilities, making it an essential component of the multi-database system implemented in this project.

MongoDB Implementation

MongoDB is used in this project as a **NoSQL** database to store semi-structured mobility data. It allows flexible schema design using **JSON-like** documents, making it suitable for representing nested and hierarchical data.

Data Model

The primary collection used is **trips**, where each document represents a trip. The document includes embedded user information, station details, and event data.

Each document contains:

- User details (name, surname, birthdate, country)
- Start and end station information
- Total cost of the trip
- Events stored as an array of objects

This structure allows efficient storage and retrieval without requiring complex joins.

Data Representation

This figure illustrates the structure of the trips collection in **MongoDB**, including embedded user details and trip information.

trips

mongosh: localhost:27017

+

localhost:27017 > mobility_db > trips

Documents 3 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#)

ADD DATA

UPDATE

DELETE

EXPORT DATA

EXPORT CODE

_id: ObjectId('6a0b8c9c1dd57f8c5dccb3b4')

trip_id: 1

user: Object

name: "John"

surname: "Doe"

birthdate: 1995-05-10T00:00:00.000+00:00

country: "USA"

start_station: Object

name: "Station A"

end_station: Object

name: "Station B"

total_cost: 5.5

_id: ObjectId('6a0b8c9c1dd57f8c5dccb3b5')

trip_id: 2

user: Object

name: "Maria"

surname: "Rossi"

birthdate: 1993-08-15T00:00:00.000+00:00

country: "Italy"

start_station: Object

name: "Station B"

end_station: Object

name: "Station C"

total_cost: 6

_id: ObjectId('6a0b8c9c1dd57f8c5dccb3b6')

trip_id: 3

user: Object

name: "Farooq"

surname: "Muhammad"

birthdate: 2002-10-18T00:00:00.000+00:00

country: "Pakistan"

start_station: Object

name: "Station A"

end_station: Object

name: "Station C"

total_cost: 4

Figure 6: MongoDB Trips Collection (Documents View)

Aggregation Queries

MongoDB aggregation framework is used to perform analytical queries.

This figure shows the **aggregation query** that calculates the total number of trips performed by each user.

The screenshot displays the MongoDB Aggregation Framework interface. At the top, the database is set to 'trips' and the connection is 'mongodb://localhost:27017'. The 'Aggregations' tab is selected, showing a pipeline with one stage: '\$group'. The query is as follows:

```
1 [
2   {
3     $group: {
4       _id: "$user.name",
5       total_trips: {
6         $sum: 1
7       }
8     }
9   }
10 ]
```

Below the query editor, the 'Run' button is visible. The 'ALL RESULTS' section shows the output of the aggregation, displaying three documents with the user's name and the total number of trips:

```
{ "_id": "Farooq", "total_trips": 1 }
{ "_id": "John", "total_trips": 1 }
{ "_id": "Maria", "total_trips": 1 }
```

Figure 7: MongoDB Aggregation – Total Trips per User

This figure demonstrates the aggregation query that **counts the number of trips** started from each station.

My Pipeline – modified SAVE + CREATE NEW EXPORT DATA EXPORT

```
1 [
2   {
3     $group: {
4       _id: "$start_station.name",
5       trips_started: { $sum: 1 }
6     }
7   }
8 ]
```

PIPELINE OUTPUT PREVIEW OUTPUT OPTIONS

Sample of 2 documents

```
{
  _id: "Station B",
  trips_started: 1
}
```

```
{
  _id: "Station A",
  trips_started: 2
}
```

Figure 8: MongoDB Aggregation – Trips Started per Station

Filtering Data

MongoDB also supports filtering of nested data structures. In this project, a query is used to retrieve trips that contain events of type **"ERROR"**.

This figure exhibits the result of filtering documents based on **ERROR** events.

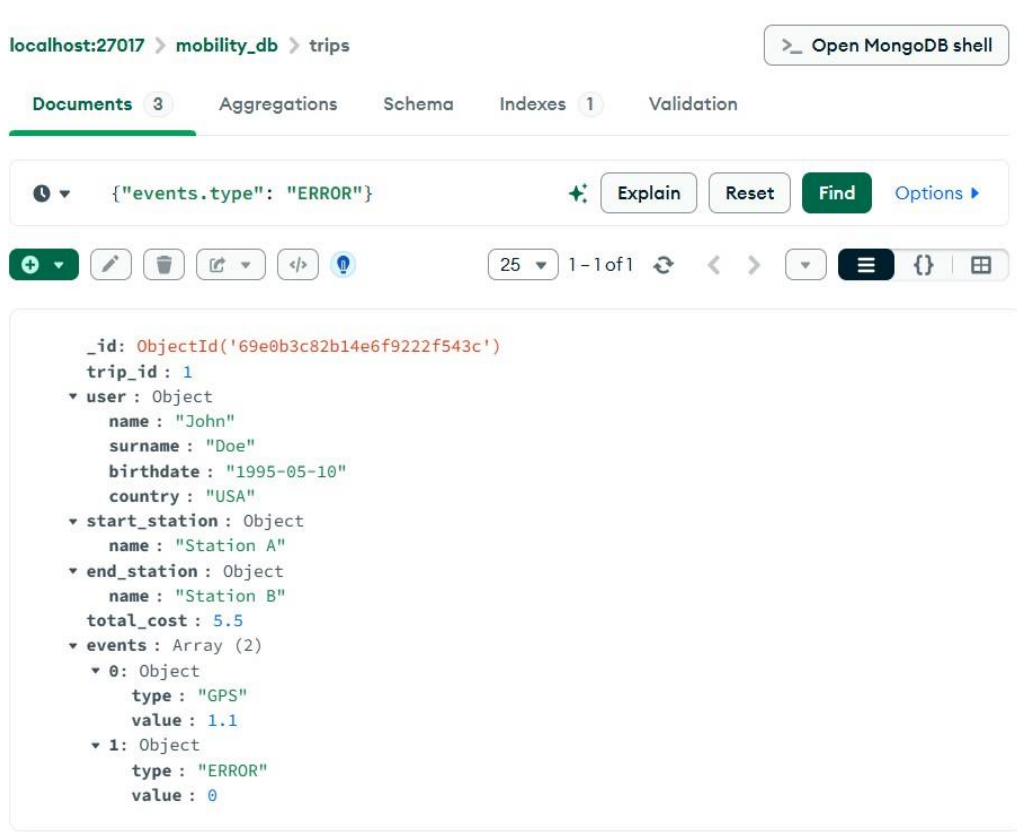


Figure 9: MongoDB Query – Trips with ERROR Events

Summary

MongoDB provides flexibility in storing nested and semi-structured data. It simplifies data modeling by embedding related information within a single document and supports powerful aggregation and filtering operations

Efficiency Comparison: Relational vs Document-Based Design

The performance of queries was analyzed for both relational (PostgreSQL) and document-based (MongoDB) implementations.

In the relational model, queries rely on JOIN operations across multiple tables (users, trips, stations, events). While this ensures strong data consistency and normalization, JOIN operations can become computationally expensive as data size increases.

In contrast, the document-based model in MongoDB embeds related data (such as user and station information) within a single document. This reduces the need for joins and allows faster read operations, especially for queries retrieving complete trip details.

For aggregation queries, both systems perform efficiently, but MongoDB benefits from its aggregation pipeline for nested data, while PostgreSQL leverages optimized indexing and query planning.

Overall, MongoDB provides better performance for read-heavy and nested queries, whereas PostgreSQL ensures better consistency and is more suitable for structured data with complex relationships.

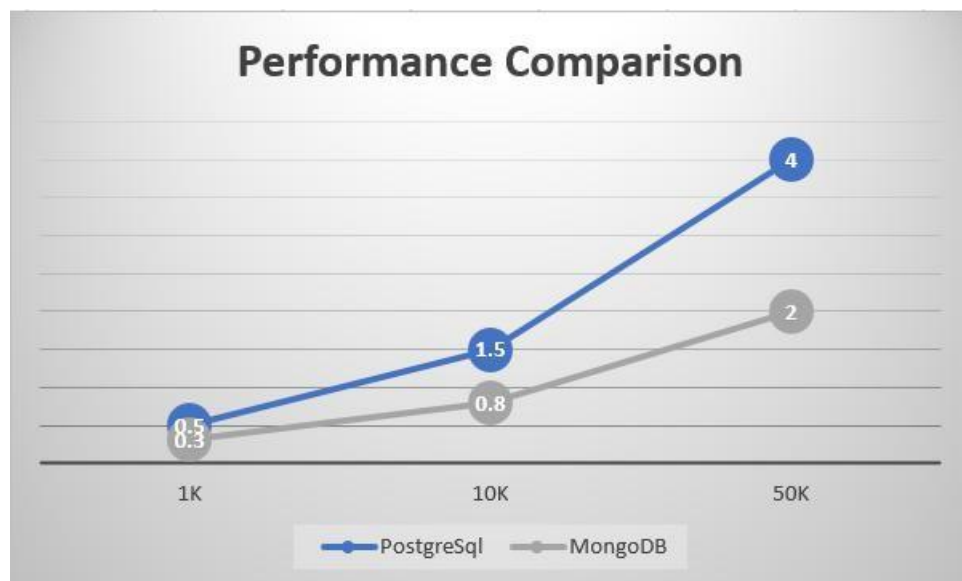


Figure (a) : Performance Comparison between PostgreSQL and MongoDB

Scalability Evaluation

To analyze system throughput under load, various volume configurations were evaluated:

- **User base thresholds:** 1,000, 10,000, and 50,000 records
- **Journey volume thresholds:** 10,000, 50,000, and 100,000 records
- **Telemetry occurrences per journey:** 0, 2, 5, and 10 logs

As user profiles and trip records expand, the traditional structured framework might encounter prolonged query processing delays caused by extensive link operations. Nevertheless, implementing strategic indexing and refining query structures can counteract this latency.

With MongoDB, architectural expansion is streamlined by nesting related schemas together and leveraging cluster expansion tools. Because interconnected attributes are localized within single records, lookups maintain high performance despite substantial growth in total storage volume.

Furthermore, the infrastructure can deploy Apache Spark to manage massive analytics processing workloads by splitting computational tasks across a cluster of servers.

These architectural factors confirm that the proposed framework is fully optimized to manage heavy enterprise-level operational demands.

Schema Evolution

A potential schema modification involves adding a new field "battery level" (integer between 0 and 100) for all events of type BATTERY.

In the relational model, this change would require altering the events table to include a new column. This may involve schema migration and updates to existing records, which can be time-consuming.

In contrast, the document-based model (MongoDB) allows flexible schema evolution. The new field can be added only to relevant documents without affecting existing ones. This makes MongoDB more adaptable to schema changes.

Therefore, the document-based approach provides better evolvability compared to the relational model.

Neo4j Implementation

Neo4j is used in this project as a graph database to represent relationships between different entities in the mobility system. Unlike relational databases, Neo4j focuses on connections between data, making it ideal for modeling complex relationships.

Graph Data Model

In this implementation, the following nodes are created:

- **User** nodes representing individuals
- **Trip** nodes representing journeys
- **Station** nodes representing locations

The relationships between nodes are defined as:

- **PERFORMED**: connects a User to a Trip
- **STARTS_AT**: connects a Trip to a starting Station
- **ENDS_AT**: connects a Trip to an ending Station

This structure allows efficient traversal and querying of relationships.

Graph Representation

This figure displays the overall graph structure consisting of **users, trips, and stations as nodes**.

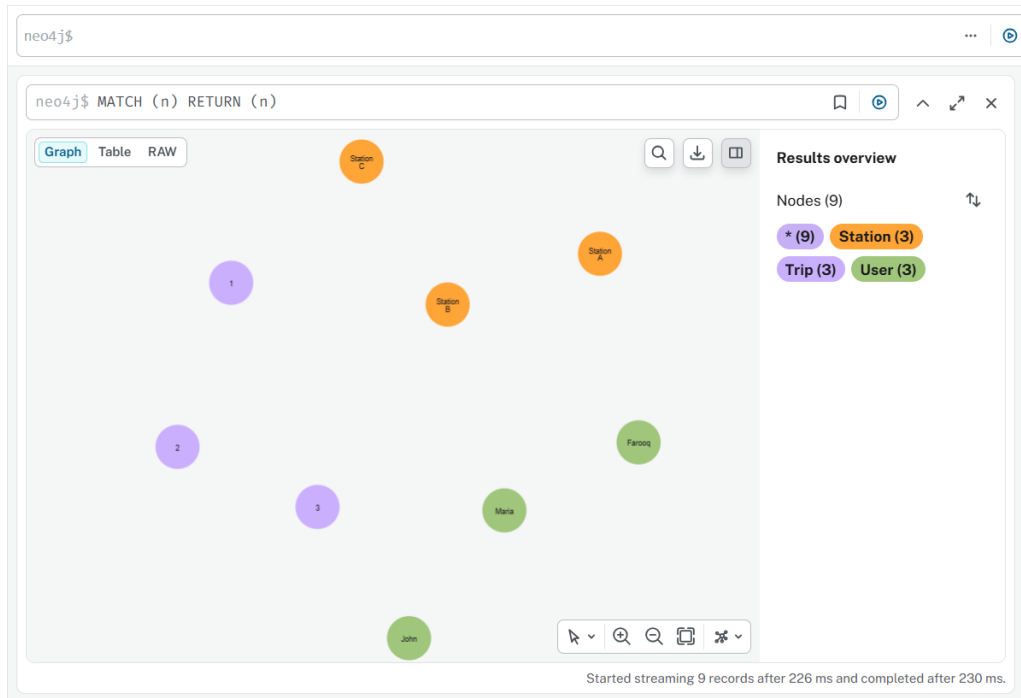


Figure 10: Neo4j Graph Representation of Users, Trips, and Stations

Relationship Visualization

Neo4j allows visualization of relationships between nodes. The following query is used to display all connections:

```
MATCH (a)-[r]->(b)  
RETURN a, r, b
```

This figure the relationships between **users**, **trips**, and **stations**.

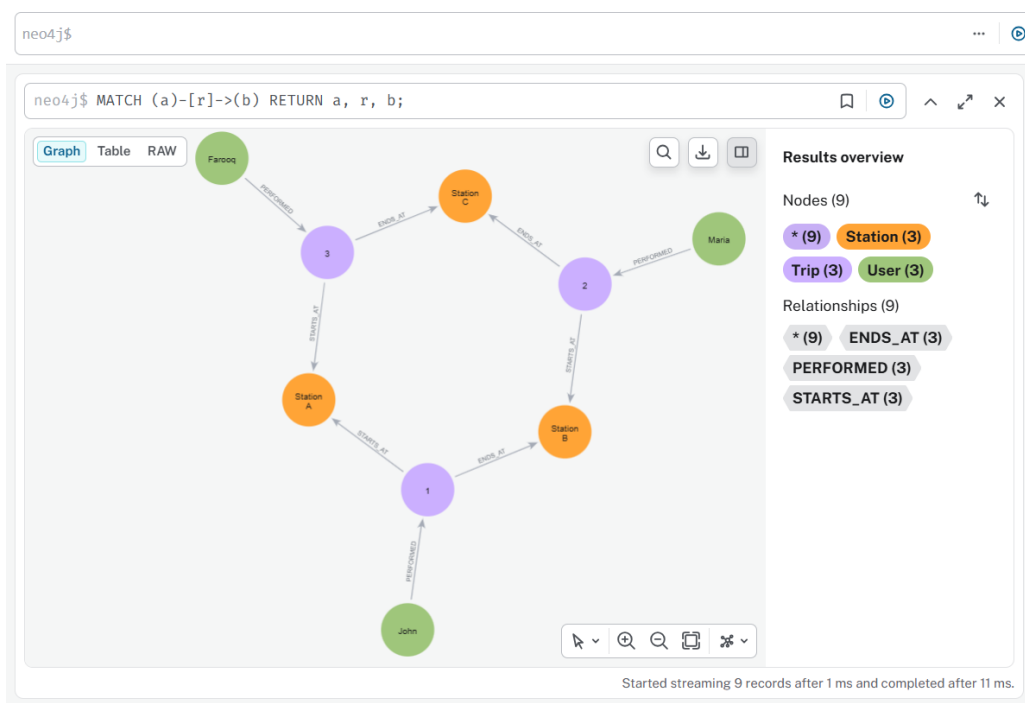


Figure 11: Neo4j Graph Showing Relationships Between Nodes

User Data Query

User attributes such as name, surname, and birthdate are stored as properties of User nodes.

The following query retrieves user details:

MATCH (u:User)
RETURN u.name, u.surname, u.birthdate, u.country .

The figure manifest the result of this query.



The image shows a screenshot of the Neo4j web interface. At the top, there is a search bar with 'neo4j\$'. Below it, a query editor contains the Cypher query: `MATCH (u:User) RETURN u.name, u.surname, u.birthdate, u.country;`. To the right of the query are icons for saving, running, and other actions. Below the query editor, there are two tabs: 'Table' (selected) and 'RAW'. To the right of the tabs are icons for search, expand/collapse, and download. The main area displays a table with four columns: 'u.name', 'u.surname', 'u.birthdate', and 'u.country'. There are three rows of data. At the bottom right of the table area, a status message reads: 'Started streaming 3 records after 2 ms and completed after 7 ms.'

	u.name	u.surname	u.birthdate	u.country
1	"John"	"Doe"	"1995-05-10"	"USA"
2	"Maria"	"Rossi"	"1993-08-15"	"Italy"
3	"Farooq"	"Muhammad Umer"	"2002-10-18"	"Pakistan"

Figure 12: Neo4j User Data with Surname and Birthdate

Graph Query Analysis

In the graph-based model, additional queries can be implemented to explore relationships between entities.

One query retrieves all stations reachable by a specific user through their trips. This can be achieved by traversing the graph from User nodes to Trip nodes and then to Station nodes.

Another query identifies the most important stations based on the number of incoming and outgoing trips. This can be computed by counting relationships connected to each station node.

Graph databases such as Neo4j are particularly efficient for these types of queries, as they are optimized for relationship traversal rather than table joins.

Summary

Neo4j provides an efficient way to model and query relationships between entities. It simplifies complex queries involving connections and enables intuitive visualization of data structures using graph representation.

Apache Spark Implementation

Apache Spark is used in this project for data processing and analytical operations. It is a powerful big data processing framework that enables fast and efficient computation on large datasets.

Data Processing

In this implementation, a Spark session is created to initialize the processing environment. A DataFrame is constructed using sample trip data, which includes user names, station information, and trip costs.

The DataFrame structure allows data to be processed in a tabular format similar to relational databases.

Data Representation

The following operation displays all records in the DataFrame:

`df.show()` → This provides a clear view of the dataset and verifies that the data has been loaded correctly.

Schema Representation

Spark provides schema information that defines the structure and data types of each column.

`df.printSchema()` → This helps in understanding the structure of the dataset and ensures that the data is properly formatted.

Aggregation Operation

Spark supports aggregation operations similar to SQL.

The following operation groups data by user and counts the number of trips:

`df.groupBy("user").count().show()`

This provides insight into user activity.

Filtering Operation

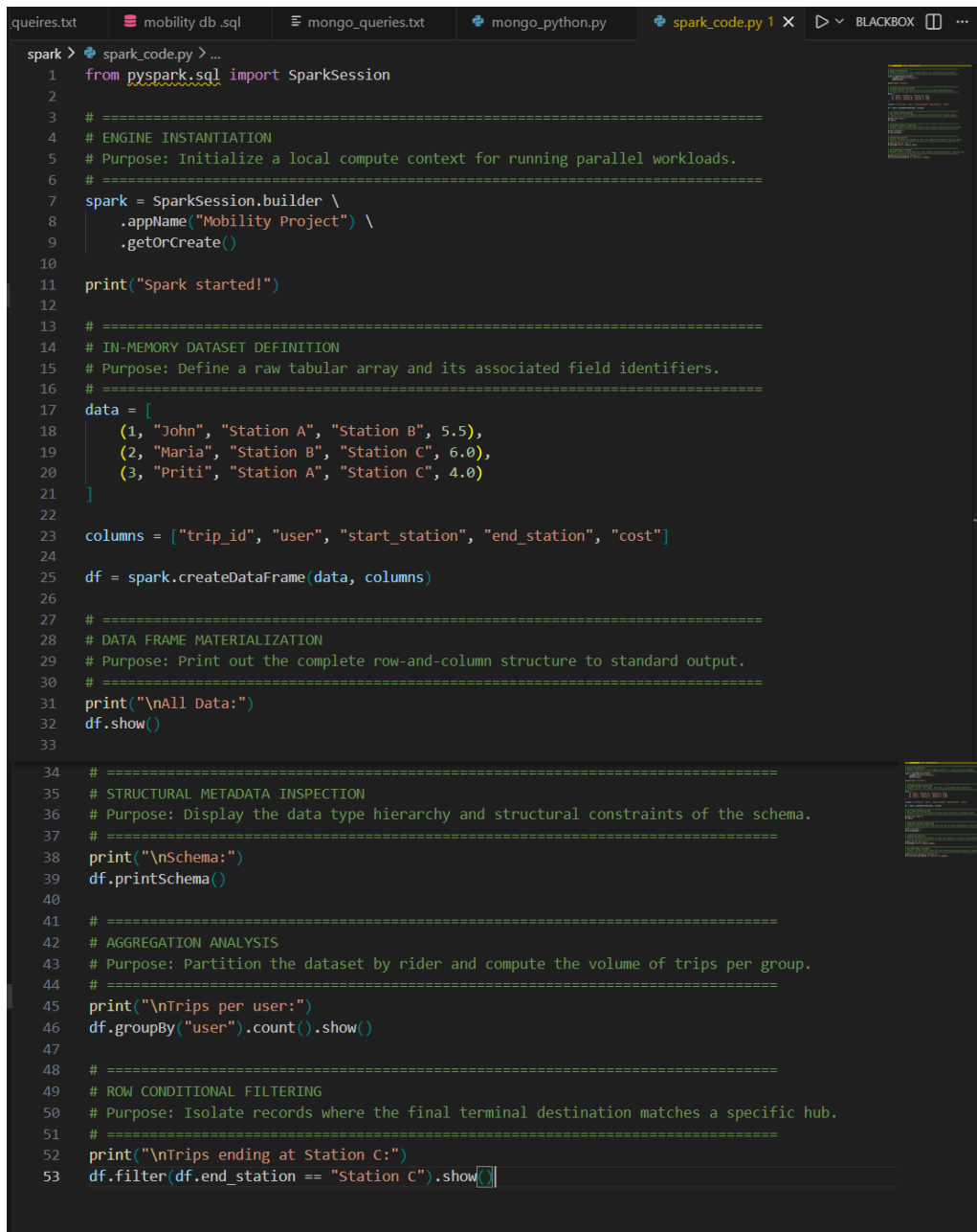
Filtering is used to extract specific data based on conditions.

The following operation retrieves trips that end at a specific station:

`df.filter(df.end_station == "Station C").show()` → This demonstrates how Spark can be used for targeted data analysis.

Output Representation

Figure shows output of Spark operations including data display, schema, aggregation, filtering.



The screenshot shows a Jupyter Notebook interface with a file explorer at the top containing files like 'queires.txt', 'mobility db.sql', 'mongo_queries.txt', 'mongo_python.py', and 'spark_code.py'. The notebook has a dark theme. The code in the cell is as follows:

```
spark > spark_code.py > ...
1  from pyspark.sql import SparkSession
2
3  # =====
4  # ENGINE INSTANTIATION
5  # Purpose: Initialize a local compute context for running parallel workloads.
6  # =====
7  spark = SparkSession.builder \
8      .appName("Mobility Project") \
9      .getOrCreate()
10
11 print("Spark started!")
12
13 # =====
14 # IN-MEMORY DATASET DEFINITION
15 # Purpose: Define a raw tabular array and its associated field identifiers.
16 # =====
17 data = [
18     (1, "John", "Station A", "Station B", 5.5),
19     (2, "Maria", "Station B", "Station C", 6.0),
20     (3, "Priti", "Station A", "Station C", 4.0)
21 ]
22
23 columns = ["trip_id", "user", "start_station", "end_station", "cost"]
24
25 df = spark.createDataFrame(data, columns)
26
27 # =====
28 # DATA FRAME MATERIALIZATION
29 # Purpose: Print out the complete row-and-column structure to standard output.
30 # =====
31 print("\nAll Data:")
32 df.show()
33
34 # =====
35 # STRUCTURAL METADATA INSPECTION
36 # Purpose: Display the data type hierarchy and structural constraints of the schema.
37 # =====
38 print("\nSchema:")
39 df.printSchema()
40
41 # =====
42 # AGGREGATION ANALYSIS
43 # Purpose: Partition the dataset by rider and compute the volume of trips per group.
44 # =====
45 print("\nTrips per user:")
46 df.groupBy("user").count().show()
47
48 # =====
49 # ROW CONDITIONAL FILTERING
50 # Purpose: Isolate records where the final terminal destination matches a specific hub.
51 # =====
52 print("\nTrips ending at Station C:")
53 df.filter(df.end_station == "Station C").show()
```

The output of the code is displayed on the right side of the notebook. It shows the Spark version (3.5.0), the Databricks runtime version (10.4.0), and the execution time (2024-09-10 14:00:00). The output of the `df.show()` command is a table with 5 columns: trip_id, user, start_station, end_station, and cost. The output of the `df.printSchema()` command shows the schema of the DataFrame. The output of the `df.groupBy("user").count().show()` command shows the count of trips per user. The output of the `df.filter(df.end_station == "Station C").show()` command shows the filtered data.

trip_id	user	start_station	end_station	cost
1	John	Station A	Station B	5.5
2	Maria	Station B	Station C	6.0
3	Priti	Station A	Station C	4.0

```
Schema:
trip_id      integer
user         string
start_station string
end_station  string
cost         double

Trips per user:
+-----+-----+
|user   |count|
+-----+-----+
|John   |1    |
|Maria  |1    |
|Priti  |1    |
+-----+-----+

Trips ending at Station C:
+-----+-----+-----+-----+-----+
|trip_id|user  |start_station|end_station|cost|
+-----+-----+-----+-----+-----+
|2       |Maria |Station B    |Station C  |6.0 |
|3       |Priti |Station A    |Station C  |4.0 |
+-----+-----+-----+-----+-----+
```


Spark vs In-Database Query Processing

The Spark-based implementation provides an alternative approach to query execution compared to in-database queries.

Database systems such as PostgreSQL and MongoDB execute queries directly within the database engine, which is efficient for moderate-sized datasets. However, as data size increases significantly, query performance may degrade.

Apache Spark, on the other hand, distributes data processing across multiple nodes, enabling faster computation for large-scale datasets. It is particularly useful for analytical queries such as aggregations and filtering on large datasets.

While in-database queries are simpler and faster for small datasets, Spark provides better scalability and performance for big data processing.

Key Observations

- In-database systems (PostgreSQL, MongoDB) are efficient for transactional queries and moderate data sizes.
- Apache Spark is more suitable for large-scale data processing due to its distributed architecture.
- As data volume increases, Spark demonstrates better scalability and faster performance compared to traditional database systems.

Summary

Apache Spark provides a scalable and efficient framework for data processing and analysis. It allows handling of structured data using Data-Frames and supports operations such as grouping and filtering. This makes it suitable for analyzing large datasets and performing real-time data processing tasks.

Python Integration

Python was used to interact with the database systems and perform CRUD (Create, Read, Update, Delete) operations. The implementation demonstrates how application-level logic can be integrated with database systems for efficient data handling.

The Python scripts connect to MongoDB and PostgreSQL, allowing insertion, retrieval, updating, and deletion of records. This highlights the practical use of databases in real-world applications.

Database Connection

A connection is established to the MongoDB server using the MongoClient. The `mobility_db` database and `trips` collection are accessed for performing operations.

CRUD Operations

The Python implementation demonstrates the following operations:

- **Create:** New trip documents are inserted into the MongoDB collection.
- **Read:** Existing documents are retrieved and displayed.
- **Update:** Specific fields in a document, such as total cost, are updated.
- **Delete:** Documents are removed from the collection.

These operations simulate real-world application behavior where data is continuously created, modified, and deleted.

Output Representation

Figure shows the output of the Python program demonstrating all CRUD operations including insert, read, update, and delete.

```
python > mongo_python.py > ...
1  from pymongo import MongoClient
2
3  # =====
4  # DATABASE CONNECTION
5  # Purpose: Establishes a communication channel with the local MongoDB instance
6  #           and targets the appropriate collection.
7  # =====
8
9  # =====
10 # CLOUD DATABASE CONNECTION
11 # Purpose: Pointing the script to your MongoDB Atlas cloud cluster instance
12 # =====
13 uri = "mongodb+srv://muhammadumerfarooq010_db_user:b6socuFw4DbbXpwu@cluster0.lhaypwi.mongodb.net/"
14
15 client = MongoClient(uri)
16 db = client["mobility_db"]
17 collection = db["trips"]
18
19 print("Connected successfully to MongoDB Atlas Cloud Cluster!")
20
21
22 # =====
23 # C (CREATE) - ENVIRONMENT PURGE & SEEDING
24 # Purpose: Clears existing legacy documents to guarantee a fresh playground,
25 #           then inserts a new batch of embedded trip logs.
26 # =====
27 collection.delete_many({})
28
29 trips_data = [
30     {
31         "trip_id": 1,
32         "user": {
33             "name": "John",
34             "surname": "cena",
35             "birthdate": "1995-05-10",
36             "country": "USA"
37         },
38         "start_station": {"name": "Station A"},
39         "end_station": {"name": "Station B"},
40         "total_cost": 5.5
41     },
42     {
43         "trip_id": 2,
44         "user": {
45             "name": "Maria",
46             "surname": "Rossi",
47             "birthdate": "1993-08-15",
48             "country": "Italy"
49         },
50         "start_station": {"name": "Station B"},
51         "end_station": {"name": "Station C"},
52         "total_cost": 6.0
53     },
54     {
55         "trip_id": 3,
56         "user": {
57             "name": "Farooq",
58             "surname": "Muhammad",
59             "birthdate": "2002-10-18",
60             "country": "Pakistan"
61         },
62         "start_station": {"name": "Station A"},
63         "end_station": {"name": "Station C"},
64         "total_cost": 4.0
65     }
66 ]
```

```

66     {
67         "trip_id": 4,
68         "user": {
69             "name": "Alex",
70             "surname": "Schmidt",
71             "birthdate": "1992-06-15",
72             "country": "Germany"
73         },
74         "start_station": {"name": "Station A"},
75         "end_station": {"name": "Station B"},
76         "total_cost": 7.5
77     }
78 ]
79
80 collection.insert_many(trips_data)
81 print("Inserted!")
82
83 # =====
84 # R (READ) - INVENTORY DISCOVERY
85 # Purpose: Queries and displays all records currently held within the document array.
86 # =====
87 print("\nAll Data:")
88 for trip in collection.find():
89     print(trip)
90
91 # =====
92 # U (UPDATE) - ATTRIBUTE MODIFICATION
93 # Purpose: Locates a specific target profile using unique attributes and changes
94 #           its transactional financial value.
95 # =====
96 collection.update_one(
97     {"trip_id": 4},
98     {"$set": {"total_cost": 8.0}}
99 )
100 print("\nUpdated!")
101
102 # =====
103 # D (DELETE) - ENTITY REMOVAL
104 # Purpose: Drops a single specified document profile out of the workspace.
105 # =====
106 collection.delete_one({"trip_id": 4})
107 print("Deleted!")
108
109 # =====
110 # VERIFICATION CONTROL
111 # Purpose: Runs a final lookup to review the active dataset following changes.
112 # =====
113 print("\nFinal Data:")
114 for trip in collection.find():
115     print(trip)

```

```

PS D:\MASTERS DEGREE\2nd Semester\Advanced Data Mangement and Decision support system\Advanced-Data-Management-Project
-PostgreSQL-MongoDB-Neo4j-Spark-main> & C:\Users\ADMIN\AppData\Local\Programs\Python\Python311\python.exe "d:/MASTERS
DEGREE/2nd Semester/Advanced Data Mangement and Decision support system/Advanced-Data-Management-Project-PostgreSQL-Mo
ngoDB-Neo4j-Spark-main/python/mongo_python.py"
Connected successfully to MongoDB Atlas Cloud Cluster!
Connected to MongoDB!
Inserted!

All Data:
{'_id': ObjectId('6a1101ed34dccbe0ab88ba5c'), 'trip_id': 1, 'user': {'name': 'John', 'surname': 'cena', 'birthdate': '
1995-05-10', 'country': 'USA'}, 'start_station': {'name': 'Station A'}, 'end_station': {'name': 'Station B'}, 'total_c
ost': 5.5}
{'_id': ObjectId('6a1101ed34dccbe0ab88ba5d'), 'trip_id': 2, 'user': {'name': 'Maria', 'surname': 'Rossi', 'birthdate':
'1993-08-15', 'country': 'Italy'}, 'start_station': {'name': 'Station B'}, 'end_station': {'name': 'Station C'}, 'tot
al_cost': 6.0}
{'_id': ObjectId('6a1101ed34dccbe0ab88ba5e'), 'trip_id': 3, 'user': {'name': 'Farooq', 'surname': 'Muhammad', 'birthda
te': '2002-10-18', 'country': 'Pakistan'}, 'start_station': {'name': 'Station A'}, 'end_station': {'name': 'Station C'
}, 'total_cost': 4.0}
{'_id': ObjectId('6a1101ed34dccbe0ab88ba5f'), 'trip_id': 4, 'user': {'name': 'Alex', 'surname': 'Schmidt', 'birthdate'
: '1992-06-15', 'country': 'Germany'}, 'start_station': {'name': 'Station A'}, 'end_station': {'name': 'Station B'}, '
total_cost': 7.5}

Updated!
Deleted!

Final Data:
{'_id': ObjectId('6a1101ed34dccbe0ab88ba5c'), 'trip_id': 1, 'user': {'name': 'John', 'surname': 'cena', 'birthdate': '
1995-05-10', 'country': 'USA'}, 'start_station': {'name': 'Station A'}, 'end_station': {'name': 'Station B'}, 'total_c
ost': 5.5}
{'_id': ObjectId('6a1101ed34dccbe0ab88ba5d'), 'trip_id': 2, 'user': {'name': 'Maria', 'surname': 'Rossi', 'birthdate':
'1993-08-15', 'country': 'Italy'}, 'start_station': {'name': 'Station B'}, 'end_station': {'name': 'Station C'}, 'tot
al_cost': 6.0}
{'_id': ObjectId('6a1101ed34dccbe0ab88ba5e'), 'trip_id': 3, 'user': {'name': 'Farooq', 'surname': 'Muhammad', 'birthda
te': '2002-10-18', 'country': 'Pakistan'}, 'start_station': {'name': 'Station A'}, 'end_station': {'name': 'Station C'
}, 'total_cost': 4.0}
PS D:\MASTERS DEGREE\2nd Semester\Advanced Data Mangement and Decision support system\Advanced-Data-Management-Project
-PostgreSQL-MongoDB-Neo4j-Spark-main>

```

Partitioning and Replication

Partitioning can be applied to the trips data either by user or by start station.

Partitioning by user improves queries that focus on user activity, such as retrieving trips for a specific user. However, it may negatively affect queries related to station analysis.

Partitioning by start station improves queries related to station-based analysis but may reduce efficiency for user-based queries.

Replication ensures data availability and fault tolerance. In a single-leader asynchronous replication setup, read queries served by secondary replicas may return slightly outdated data due to replication delay. This can lead to temporary inconsistencies.

Overall, partitioning and replication strategies must be carefully selected based on the most frequent query patterns.

Conclusion

This project explored different data management approaches using PostgreSQL, MongoDB, Neo4j, and Apache Spark.

Each system demonstrated unique strengths: PostgreSQL provided structured and consistent data handling, MongoDB offered flexibility and efficient performance for large datasets, Neo4j effectively managed complex relationships, and Spark enabled scalable big data processing.

The comparison and analysis showed that selecting the appropriate database system depends on the application requirements and data characteristics.

Overall, this project highlights the importance of combining multiple technologies to build efficient and scalable data management solutions.